



Celeris

SysMLv2 Mermaid Viewer

Version 0.0.1 · User Manual

Features

- Render the contents of a SysML v2 package as a Mermaid diagram from the editor (right-click → Celeris).
- Five diagram types: flowchart, class, state, sequence and requirement.
- In-process parsing with the open-source SysML v2 parser (syside-languageserver); reusable profile libraries in sibling files are resolved automatically.
- Per-panel toolbar: ↻ Refresh, ↓ PNG export, zoom/pan controls (–, 1:1, +, Fit, mouse-wheel zoom, drag-to-pan), and a syntax-error indicator.
- Auto-refresh open diagrams when the .sysml file is saved (configurable).
- Type-based styling from a JSON style file: colour elements by native SysML type and/or by the SysML type they conform to (hierarchy-followed).
- Flowchart colour code by node kind by default (actions blue, control nodes amber, parts green), overridable by a style file.
- Performers in flowcharts: a part that performs an action is shown as a component linked to it by a dashed allocation edge (which component realises each function).
- Capella-style functional chains: model each chain as a use case that performs its functions to highlight the chain (and its flows) with bold coloured borders, with shared functions marked.
- Ready-made Arcadia and NAF v4 styling profiles.
- Open several diagrams at once, one per panel.

Highlights

- **Flowchart.** Action flows with decisions, fork/join/merge and guarded transitions.
- **Class / State / Sequence.** Three further diagram types sharing the parser and rendering infrastructure.
- **Auto-refresh & PNG export.** Live refresh on save and one-click PNG export of any diagram.
- **Type-based styling.** JSON style files, with reusable Arcadia and NAF v4 profiles.

Examples

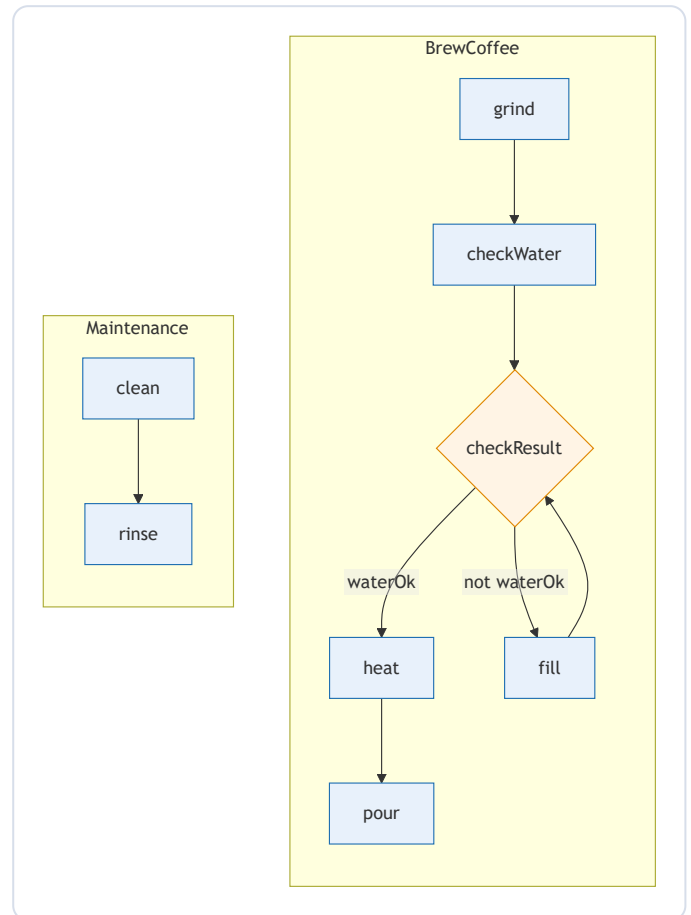
Flowchart — action flow with a decision

An action def with a decision node, guarded branches and a loop. Action usages become nodes, successions become edges, and transition guards label the edges. Non-flow elements (attributes, parts) are filtered out.

EXAMPLES/COFFEE.SYSML

```
package CoffeeMachine {  
    private import ScalarValues::*;  
  
    action def BrewCoffee {  
        attribute waterOk : Boolean;  
  
        action grind;  
        action checkWater;  
        decide checkResult;  
        action heat;  
        action fill;  
        action pour;  
  
        succession first grind then checkWater;  
        succession first checkWater then  
        checkResult;  
        first checkResult if waterOk then heat;  
        first checkResult if not waterOk then fill;  
        succession first fill then checkResult;  
        succession first heat then pour;  
    }  
  
    action def Maintenance {  
        action clean;  
        action rinse;  
        succession first clean then rinse;  
    }  
  
    part def Tank;  
    attribute level : ScalarValues::Integer;  
}
```

FLOWCHART DIAGRAM



Flowchart — parallel actions (fork / join)

`fork` and `join` control nodes split work into parallel branches and re-synchronize them.

EXAMPLES/ORDER_FULFILLMENT.SYSML

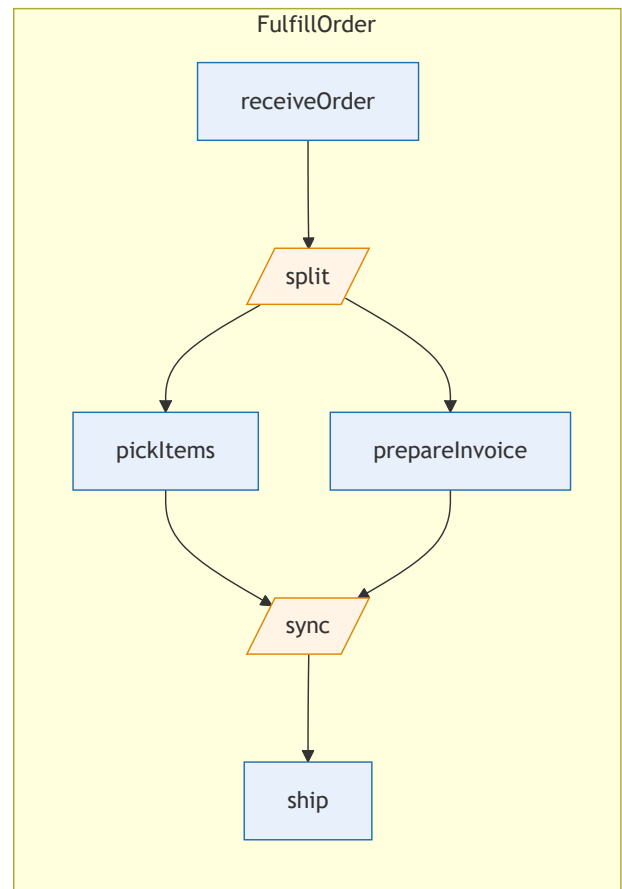
```
// Flowchart example: parallel actions with fork /
join.
package OrderFulfillment {

    action def FulfillOrder {
        action receiveOrder;
        fork split;
        action pickItems;
        action prepareInvoice;
        join sync;
        action ship;

        succession first receiveOrder then split;
        succession first split then pickItems;
        succession first split then prepareInvoice;
        succession first pickItems then sync;
        succession first prepareInvoice then sync;
        succession first sync then ship;

    }
}
```

FLOWCHART DIAGRAM



Flowchart — functional chains & performers (Capella-style)

Each chain is a use case that performs its functions; the boxes and flows of a chain get a bold coloured border, and a function shared by several chains (here **identify**) gets a dark border. The **part** performers (green) are linked to the actions they realise by dashed *performed by* edges. Actions (blue) and parts (green) are colour-coded by default. Legends recall both codes.

EXAMPLES/CHAINS/SURVEILLANCE.SYSML

```
// Functional architecture with 3 functional chains
(Capella-style).
// View as a Flowchart (right-click → Celeris →
Show Flowchart Diagram):
// Engagement, Reporting and Archival each run
through several functions; the
// 'identify' function is shared by all three
(dark border).
//
// Chains are modelled the SysML v2 way: each chain
is a 'use case' that
// 'perform's the functions it traverses. A
function performed by several use
// cases is the one shared by several chains. The
'part's are the performers:
// the components that perform (realise) each
function.
package SurveillanceArchitecture {
```

```
// — Functional architecture: the functions
and the flows between them —
action MissionThread {
    action sense;
    action detect;
    action identify;
    action engage;
    action report;
    action archive;

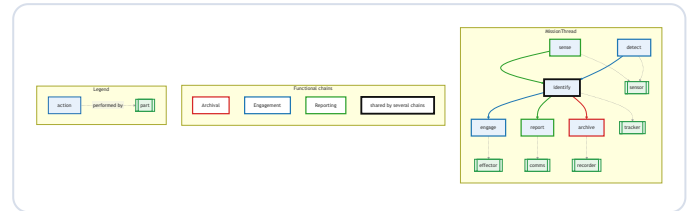
    // Functional exchanges
    succession first sense then identify;
    succession first detect then identify;
    succession first identify then engage;
    succession first identify then report;
    succession first identify then archive;
}

// — Functional chains: each chain is a use
case performing its functions —
use case def Engagement {
    perform MissionThread.detect;
    perform MissionThread.identify;
    perform MissionThread.engage;
}
use case def Reporting {
    perform MissionThread.sense;
    perform MissionThread.identify;
    perform MissionThread.report;
}
use case def Archival {
    perform MissionThread.identify;
    perform MissionThread.archive;
}

// — Performers: the components that perform
(realise) each function —
part def Sensor;
part def Tracker;
part def Effector;
part def Comms;
part def Recorder;

part sensor : Sensor {
    perform MissionThread.sense;
    perform MissionThread.detect;
}
part tracker : Tracker {
    perform MissionThread.identify;
}
part effector : Effector {
    perform MissionThread.engage;
}
```

FLOWCHART DIAGRAM



```

part comms : Comms {
  perform MissionThread.report;
}
part recorder : Recorder {
  perform MissionThread.archive;
}
}

```

Class — structure, inheritance & composition

Definitions become classes; specializations (`>`) are inheritance, typed part usages are compositions with multiplicities, and enumerations list their literals.

EXAMPLES/VEHICLE.SYSML

```

package VehicleModel {

  part def Vehicle {
    attribute mass : ScalarValues::Real;
    part engine : Engine;
    part wheels : Wheel[4];
  }

  part def Engine {
    attribute power : ScalarValues::Real;
    part cylinders : Cylinder[1..12];
  }

  part def Cylinder;
  part def Wheel;

  part def Car > Vehicle {
    attribute doors : ScalarValues::Integer;
  }

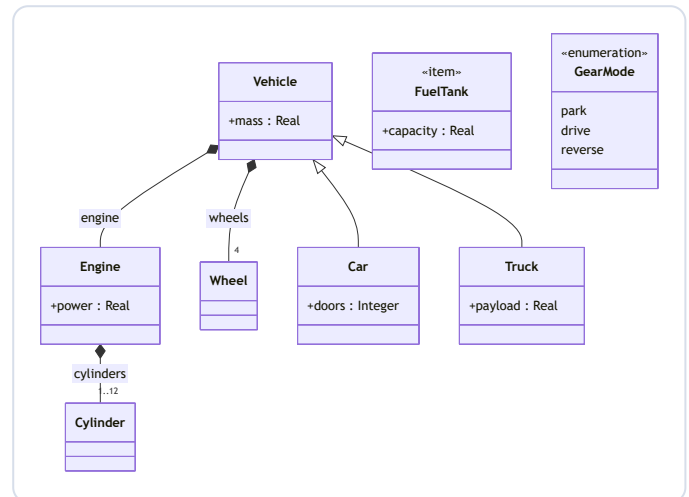
  part def Truck specializes Vehicle {
    attribute payload : ScalarValues::Real;
  }

  item def FuelTank {
    attribute capacity : ScalarValues::Real;
  }

  enum def GearMode {
    park;
    drive;
    reverse;
  }
}

```

CLASS DIAGRAM



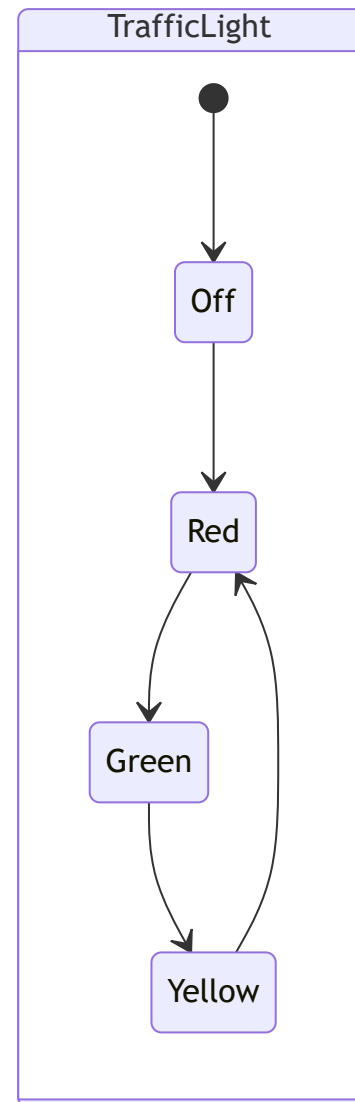
State — a traffic-light machine

A `state def` with its states and transitions. The `entry` pseudostate is rendered as the initial transition [*].

EXAMPLES/TRAFFIC_LIGHT.SYSML

```
package TrafficLightSM {  
  state def TrafficLight {  
    entry; then Off;  
  
    state Off;  
    state Red;  
    state Green;  
    state Yellow;  
  
    transition first Off then Red;  
    transition first Red then Green;  
    transition first Green then Yellow;  
    transition first Yellow then Red;  
  }  
}
```

STATE DIAGRAM



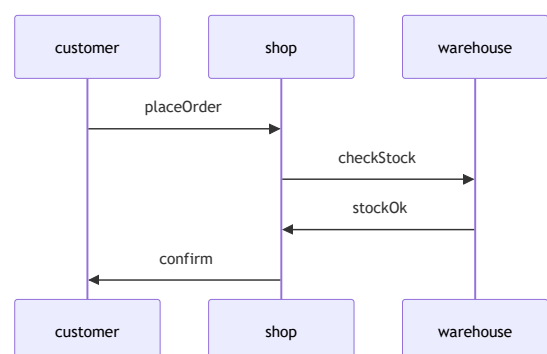
Sequence — message flow

Part usages become participants and directed `flows` become ordered messages between them.

EXAMPLES/ORDER_PROTOCOL.SYSML

```
package OrderProtocol {  
  part def Customer;  
  part def Shop;  
  part def Warehouse;  
  
  part customer : Customer;  
  part shop : Shop;  
  part warehouse : Warehouse;  
  
  flow placeOrder from customer to shop;  
  flow checkStock from shop to warehouse;  
  flow stockOk from warehouse to shop;  
  flow confirm from shop to customer;  
}
```

SEQUENCE DIAGRAM



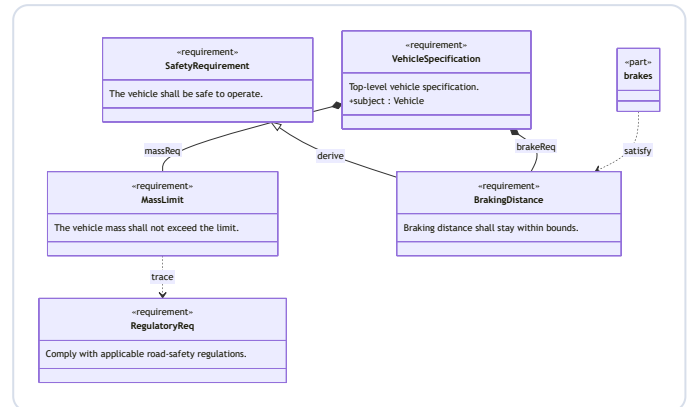
Requirement — derive, decompose & satisfy

Requirement definitions become «requirement» boxes. Specialization is derivation, nested requirement usages are decomposition, and a part that **satisfys** a requirement is linked with a satisfy dependency. The subject is shown inside the box.

EXAMPLES/VEHICLE_REQUIREMENTS.SYSML

```
package VehicleRequirements {  
  private import ScalarValues::*;  
  
  part def Vehicle;  
  part def Brakes;  
  
  requirement def RegulatoryReq {  
    doc /* Comply with applicable road-safety  
    regulations. */  
  }  
  
  requirement def SafetyRequirement {  
    doc /* The vehicle shall be safe to  
    operate. */  
  }  
  
  requirement def MassLimit {  
    doc /* The vehicle mass shall not exceed  
    the limit. */  
    attribute maxMass : Real;  
  }  
  
  requirement def BrakingDistance :>  
  SafetyRequirement {  
    doc /* Braking distance shall stay within  
    bounds. */  
  }  
  
  requirement def VehicleSpecification {  
    doc /* Top-level vehicle specification. */  
    subject vehicle : Vehicle;  
    requirement massReq : MassLimit;  
    requirement brakeReq : BrakingDistance;  
  }  
  
  // trace: the mass limit traces to a regulatory  
  requirement  
  dependency from MassLimit to RegulatoryReq;  
  
  part brakes : Brakes {  
    satisfy BrakingDistance;  
  }  
}
```

REQUIREMENT DIAGRAM



Styling — Arcadia colour code

With the Arcadia style file selected, components are coloured by their Arcadia type, following the specialization hierarchy: operational entities orange, logical components blue, physical nodes yellow.

EXAMPLES/ARCADIA/DRONE_SYSTEM.SYSML

```
// Arcadia-styled example: a small drone
surveillance system.
// View as a Class diagram with celeris.styleFile
pointing at
// examples/arcadia/arcadia.style.json to get the
Arcadia colour code:
// LogicalComponent → blue, PhysicalNode →
yellow, OperationalEntity → orange,
// functions (actions) → green.
package DroneSystem {
  private import ArcadiaProfile::*;

  // --- Operational entities (orange) ---
  part def Operator :> OperationalEntity;
  part def AirspaceAuthority :>
OperationalEntity;

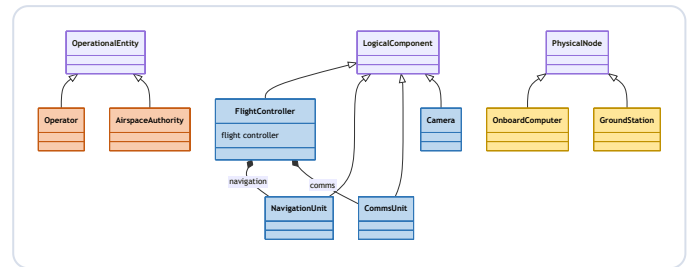
  // --- Logical components (blue) ---
  part def FlightController :> LogicalComponent {
    doc /* flight controller */
    part navigation : NavigationUnit;
    part comms : CommsUnit;
  }
  part def NavigationUnit :> LogicalComponent;
  part def CommsUnit :> LogicalComponent;
  part def Camera :> LogicalComponent;

  // --- Physical nodes (yellow) ---
  part def OnboardComputer :> PhysicalNode;
  part def GroundStation :> PhysicalNode;

  // --- Logical functions (green) ---
  action def Stabilize :> LogicalFunction;
  action def StreamVideo :> LogicalFunction;

  // --- System assembly ---
  part drone : FlightController {
    part eye : Camera;
  }
  part computer : OnboardComputer;
  part ground : GroundStation;
  part pilot : Operator;
}
```

CLASS DIAGRAM



Styling — NAF v4 by grid layer

With the NAF style file selected, elements are coloured by NAF grid layer: Concepts purple, Service blue, Logical green, Physical/Resource orange.

EXAMPLES/NAF/C2_SYSTEM.SYSML

```
// NAF v4 styled example: a small Command & Control
architecture spanning the
// NAF grid layers. View as a Class diagram with
the NAF style file selected
// (SysML Mermaid: Select Style File... →
examples/na/naf.style.json):
// Capability → purple, Service → blue, Logical
→ green, Physical → orange.
package C2System {
    private import NafProfile::*;

    // --- Concepts layer: capabilities (purple) ---
    -
    part def SituationalAwareness :> Capability;
    part def CommandAndControl :> Capability;

    // --- Service layer (blue) ---
    part def TrackingService :> Service;
    part def MessagingService :> Service;

    // --- Logical layer (green) ---
    part def SensorNode :> LogicalNode;
    part def CommandPost :> LogicalNode;

    // --- Physical / Resource layer (orange) ---
    part def Radar :> PhysicalResource;
    part def Server :> PhysicalResource;

    // --- Architecture assembly ---
    part c2 : CommandAndControl {
        part awareness : SituationalAwareness;
    }
    part tracking : TrackingService;
    part messaging : MessagingService;
    part post : CommandPost {
        part sensors : SensorNode[*];
    }
    part radar : Radar;
    part server : Server;
}
```

CLASS DIAGRAM

